



Tarea Práctica No.2

Pruebas de desempeño del algoritmo de PSO con peso de inercia.

Instrucciones:

1. Considerando las siguientes funciones de prueba (Benchmark functions) y configuraciones recomendadas para el PSO:

a) Función Suma Cuadrática con centro desplazado

- Valor máximo: 1000
- Vector solución $X_i = \{-7, -3, 3, 5, -2.5, 10, 15, -10, -15, 0.5\}$
- Espacio de búsqueda $-20 \leq x_i \leq 20$
- Sintonización: 30 partículas, 300 iteraciones, Peso de Inercia $W=0.6$

En código:

```
//Funcion Objetivo Suma Cuadrática
#define DIM 10
const unsigned int NumeroParticulas=30; //Tamaño del enjambre, numero de particulas del enjambre
const unsigned int Dimension=DIM; //Numero de variables del problema o dimension del problema.
const float LimiteSuperior[DIM]={ 20, 20, 20, 20, 20, 20, 20, 20, 20, 20};
const float LimiteInferior[DIM]={-20, -20, -20, -20, -20, -20, -20, -20, -20, -20};
const unsigned int NumeroMaximoDeIteraciones=300;

float FuncionObjetivo(float *Xi)
{float fit;
  fit=1000-pow(Xi[0]+7,2)
        -pow(Xi[1]+3,2)
        -pow(Xi[2]-3,2)
        -pow(Xi[3]-5,2)
        -pow(Xi[4]+2.5,2)
        -pow(Xi[5]-10,2)
        -pow(Xi[6]-15,2)
        -pow(Xi[7]+10,2)
        -pow(Xi[8]+15,2)
        -pow(Xi[9]-0.5,2);
  return fit;
}
```

b) Función Rastrigin de dos dimensiones

- La función se convierte en maximización multiplicando por menos uno
- Valor máximo: $F(X_i)=0.0$
- Vector solución: $X_i=\{0.0,0.0\}$
- Espacio de búsqueda: $-5.12 \leq x_i \leq 5.12$
- Sintonización: 30 partículas, 300 iteraciones, Peso de Inercia $W=0.6$

En código:

```
//Funcion Objetivo Rastrigin de Dos Dimensiones
#define DIM 2
const unsigned int NumeroParticulas=30; //Tamaño del enjambre, numero de particulas del enjambre
const unsigned int Dimension=DIM; //Numero de variables del problema o dimension del problema.
const float LimiteSuperior[DIM]={ 5.12, 5.12};
const float LimiteInferior[DIM]={-5.12, -5.12};
const unsigned int NumeroMaximoDeIteraciones=300;
```



```
float FuncionObjetivo(float *Xi)
{float fit,aux=0;
  for(unsigned int i=0; i<Dimension; i++)
    aux+=(Xi[i]*Xi[i])-10*cos(2*3.14159*Xi[i]);
  fit=-1.0*((10*2)+aux);
  return fit;
}
```

c) Función Schwefel de dos dimensiones

- La función se convierte en maximización multiplicando por menos uno
- Valor máximo: $F(x_i)=0.0$
- Vector solución: $X_i=\{420.9687, 420.9687\}$
- Espacio de búsqueda: $-500 \leq x_i \leq 500$
- Sintonización: 30 partículas, 300 iteraciones, Peso de Inercia $W=0.5$

En código:

```
//Función Objetivo Schwefel de Dos Dimensiones
#define DIM 2
const unsigned int NumeroParticulas=30; //Tamaño del enjambre, numero de particulas del enjambre
const unsigned int Dimension=DIM; //Numero de variables del problema o dimension del problema.
const float LimiteSuperior[DIM]={ 500, 500};
const float LimiteInferior[DIM]={-500, -500};
const unsigned int NumeroMaximoDeIteraciones=300;

float FuncionObjetivo(float *Xi, unsigned int Np)
{float fit,aux=0;
  if( ( (Xi[0]<-500)|| (Xi[0]>500) ) || ( (Xi[1]<-500) || (Xi[1]>500) ) )
    fit=-1500;
  else
    {for(unsigned int i=0; i<Np; i++)
      aux+=Xi[i]*sin(sqrt(fabsf(Xi[i])));
      fit=-1.0*((418.9829*2)-aux ); }
  return fit;
}
```

d) Función Shekel's Foxholes de dos dimensiones

- La función se convierte a maximización multiplicando por menos uno
- Valor máximo: $F(X_i)=0.998004$
- Vector solución: $X_i=\{-32,-32\}$
- Espacio de Búsqueda $-65.536 \leq x_i \leq 65.536$
- Sintonización: 30 partículas, 300 iteraciones, Peso de Inercia $W=0.6$

En código:

```
//Funcion Shekel's Foxholes de Dos Dimensiones
#define DIM 2
const unsigned int NumeroParticulas=60; //Tamaño del enjambre, numero de particulas del enjambre
const unsigned int Dimension=DIM; //Numero de variables del problema o dimension del problema.
const float LimiteSuperior[DIM]={ 65.536, 65.536};
const float LimiteInferior[DIM]={-65.536, -65.536};
const unsigned int NumeroMaximoDeIteraciones=300;
```



```
float FuncionObjetivo(float *Xi)
{float fit,aux1,aux2;
 int a[2][25]=
{-32,-16, 0, 16, 32,-32,-16, 0, 16, 32,-32,-16, 0, 16,32,-32,-16, 0,16,32,-32,-16, 0,16,32,-32,-32,-32,-32,-32,-16,-16,-16,-16,-16, 0, 0, 0, 0, 0, 16, 16,16,16,16, 32, 32,32,32,32};
 int i,j;
 aux1=0;
 for(j=0; j<25; j++)
 {for(i=0,aux2=0; i<2; i++)
 aux2+=pow(Xi[i]-a[i][j],6);
 aux1+=(1.0/((j+1)+aux2));
 }
 fit=1/((0.002)+aux1);
 //printf("\nf(%f,%f)=%f",Xi[0],Xi[1],fit);
 return (-1.0*fit);
}
```

Para cada función ejecutar el código 100 veces, calcular el Error Cuadrático para cada ejecución del algoritmo, almacenar en un vector columna cada uno de los cien resultados. Con los resultados elaborar el gráfico de Box-plot.

2. Analizar los resultados obtenidos en los box-plots para cada función, y describir dichos resultados por escrito.
3. Elaborar su Reporte en Formato libre y entregarlo en PDF.